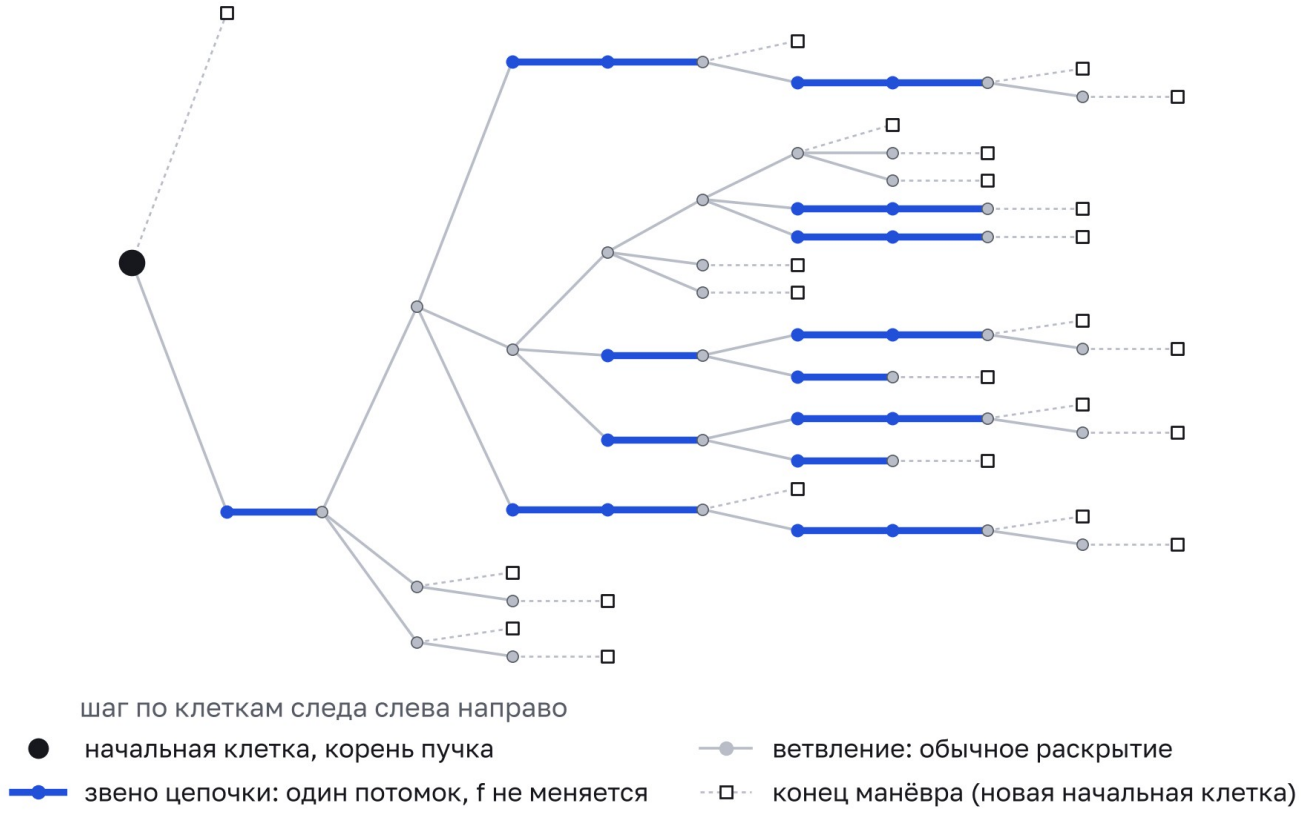
Раздел: сжатие цепочек

Развёртка пучка манёвров — дерево, и это ускоряет MeshA*

Доказываем три утверждения, сравниваем две реализации идеи на одном стенде и объединяем лучшее из обеих

Под каждой начальной клеткой — дерево; в нём 42% внутренних клеток имеют одного потомка



Что показано

Пучок курса θ = 0 базового набора: 24 манёвра, 51 внутренняя клетка.

На что смотреть

Синие звенья — клетки с единственным неначальным потомком. По ним f не меняется, поэтому их можно пройти подряд, не кладя в OPEN.

Обозначения

Серые — ветвления, квадраты — концы манёвров (новые начальные клетки).

Три утверждения дают корректность любой склейки; две леммы проверены на всей таблице переходов

Утверждение	Почему верно	Проверка
Лемма 1. У неначальной клетки ровно один предок	конфигурация = курс + общий префикс следа; по ней однозначно восстанавливается предок	864 из 864
Лемма 2. На звене цепочки f не меняется	один потомок ⇒ те же манёвры с k + 1 ⇒ те же концы ⇒ g и h те же	364 из 364
Теорема. Сжатый поиск оптимален и полон	пути и стоимости те же, h остаётся согласованной, цель не внутри цепочки	0 расхождений

Следствие леммы 1: дубликатов неначальных клеток не бывает, поэтому CLOSED нужен только начальным клеткам: 309 МиБ → 2,7 МиБ.

Связь с кодом: в MeshAStar_gluing склейка включается только после проверки ровно этих условий (один предок, совпадение Finals). Леммы объясняют, почему проверка всегда проходит.

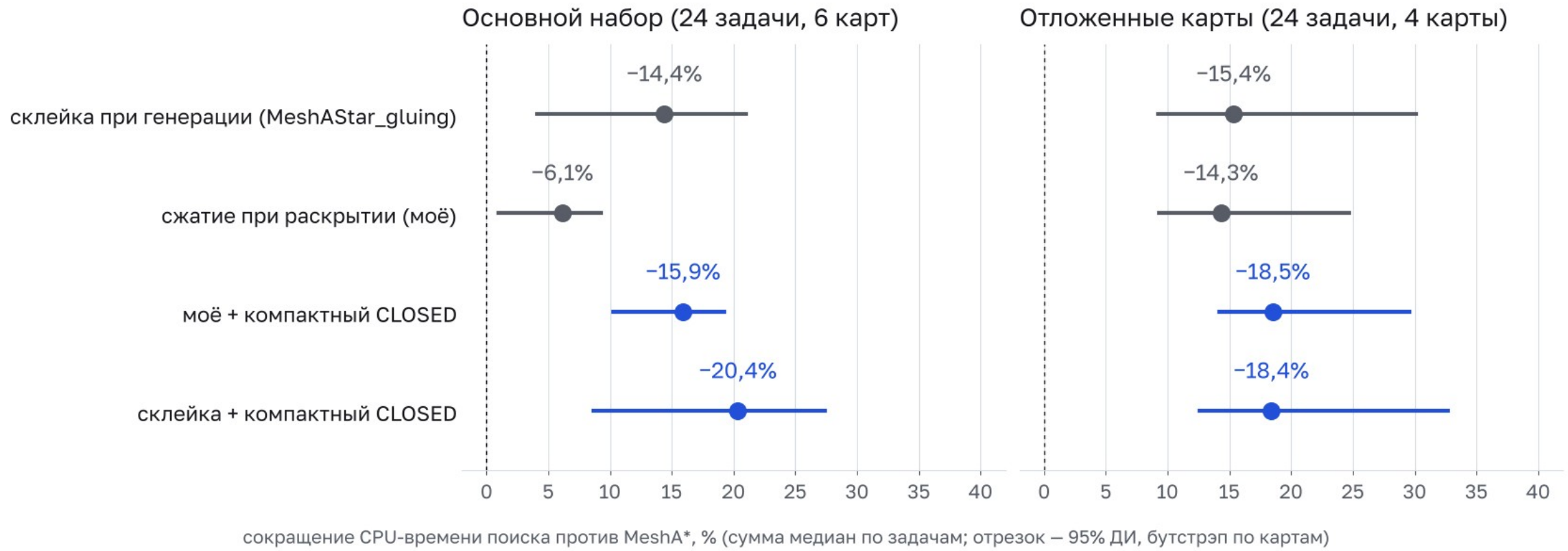
Одна идея, две реализации: склеиваем при генерации кандидата или при раскрытии вершины

	Склейка (MeshAStar_gluing)	Сжатие (моё)
Когда	до вставки потомка в OPEN	после извлечения вершины из OPEN
Что кладётся в OPEN	конец цепочки — одна вершина	потомки конца цепочки
Дополнительно	один оставшийся манёвр сразу до конца (w = 1)	компактный CLOSED (следствие леммы 1)
Подготовка	таблица путей + проверка условий лемм	таблица прыжков, строится один раз
Стоимость пути	сохраняется	сохраняется

Обе версии проверяют каждую клетку цепочки на препятствия и не перепрыгивают начальные клетки. Отличие только в моменте склейки и в CLOSED.

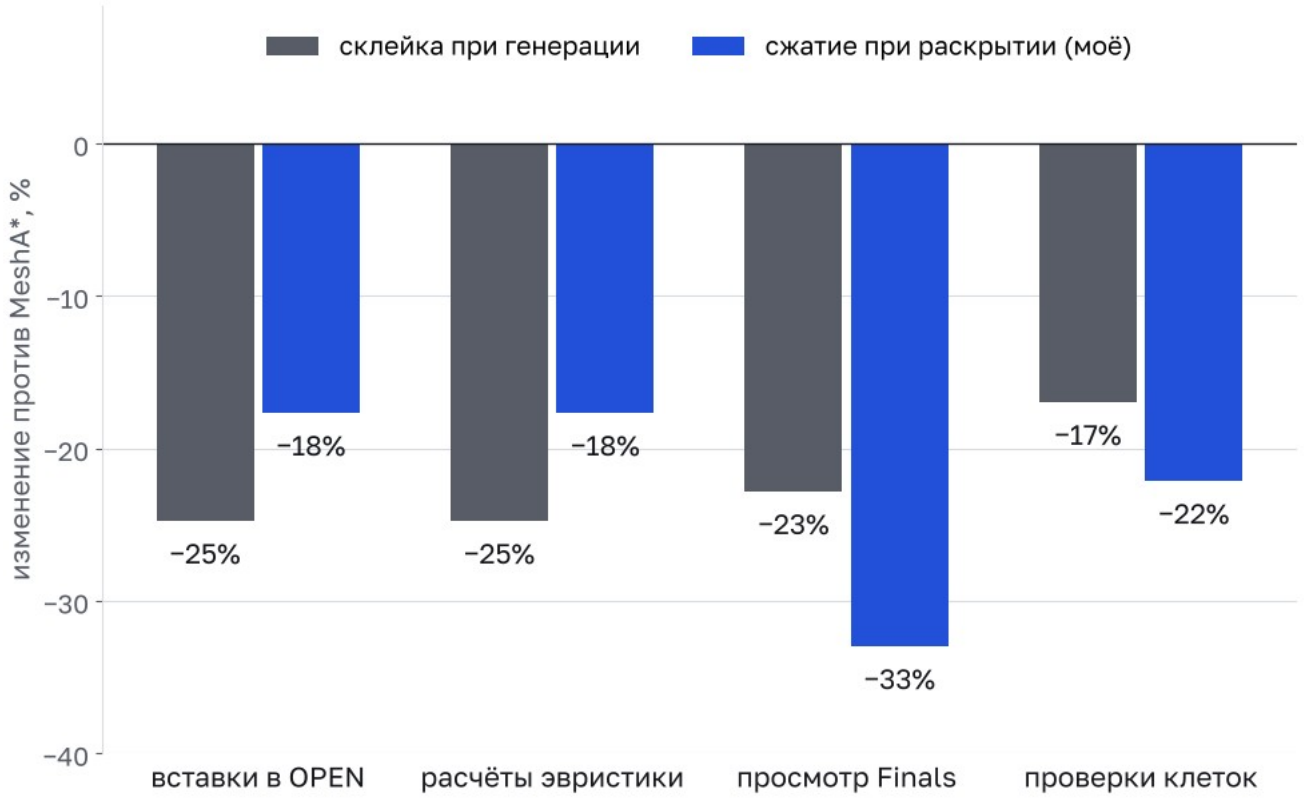
На одном стенде лучшая комбинация — склейка + компактный CLOSED: −20,4% и −18,4% времени

Стенд bench.cpp из MeshAStar_gluing: каждый процесс однопоточный и закреплён за своим ядром (2 процесса одновременно), CPU-время потока; сумма медиан по задачам, отрезок — 95% ДИ (бутстрэп по картам).



Основной набор — 2 сеанса × 5 замеров, отложенные карты — 3 замера; w = 1, наборы base и add. Linux-ВМ, 4 ядра; прогон run_compare.py из репозитория meshastar-gluing.

Склейка экономит больше вставок в OPEN, сжатие — больше просмотров Finals



Что показано

Сумма операций по 24 задачам основного набора, отдельный прогон со счётчиками.

Почему склейка быстрее

Она кладёт в OPEN одну вершину на цепочку: вставок −25% против −18%. Куча — самая дорогая структура поиска.

Почему CLOSED помогает обеим

Битовая маска 309 МиБ почти не помещается в кэш; 2,7 МиБ помещается. Это даёт ещё 3–10 п. п. времени.

Склейка с компактным CLOSED корректна и устойчиво быстрее исходного MeshA*

−20,4%

времени на основном наборе

24 задачи, 10 замеров, быстрее в 24 из 24, 95% ДИ −9…−27%

0

расхождений стоимости пути

48 задач стенда и 172 задачи широкого прогона при w = 1

2,7 МиБ

CLOSED вместо 309 МиБ

следствие леммы 1: неначальные клетки не повторяются

Время поиска без загрузки карты и создания CLOSED. Данные и скрипты: meshastar-gluing/research/gluing (run_compare.py, results/linux_compare).

Итог: теория обосновывает склейку, а её лучший вариант ускоряет поиск на 18–20%

Что получилось

Доказано: пучок — дерево, f постоянно на цепочке, сжатый поиск оптимален и полон.

Склейка при генерации: −14,4% / −15,4%

Сжатие при раскрытии: −6,1% / −14,3%

Склейка + компактный CLOSED: −20,4% / −18,4%

основной набор / отложенные карты

Ограничения и что дальше

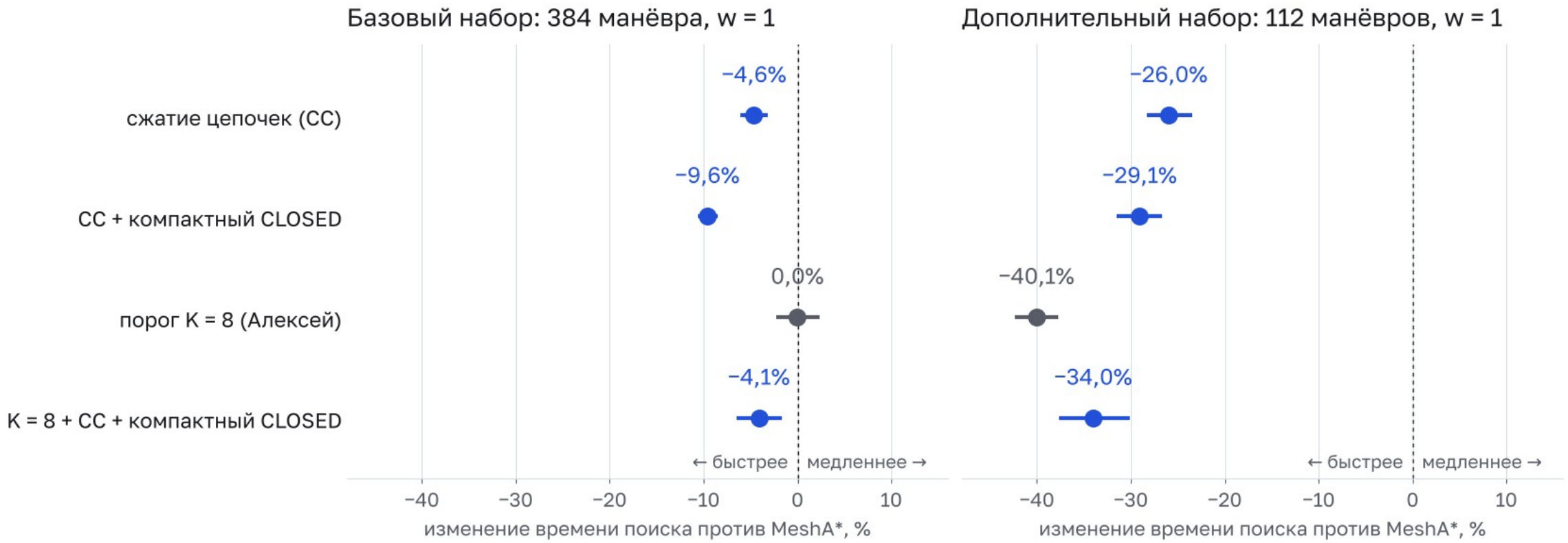
Интервалы широкие: карт всего 6 и 4, задачи без пути дают большую часть времени.

Проверок клеток в сумме меньше, но цепочка проверяется сразу целиком — при дорогой проверке это надо учитывать.

Дальше: лемма 3 (f не убывает вниз по дереву) позволяет обрабатывать вершины дерева без общей кучи.

CA. Широкий прогон моего варианта: −9,6% на базовом наборе, −29,1% на дополнительном

270 задач, 10 карт, два набора манёвров; типичная задача — среднее геометрическое отношений времени, 95% ДИ бутстрэп по задачам.



Мой стенд ccbench: CPU-время поиска, минимум из 2–3 повторов, 3 процесса параллельно. K = 8 — порог Алексея в моей реализации.

СВ. Условия двух экспериментов

	Стенд склейки (сравнение)	Широкий прогон (мой)
Задачи	core.tsv 24 + holdout.tsv 24, выбраны заранее	10 задач на карту по .scen, 270 всего
Наборы манёвров	base и add вперемешку	base (191) и add (79) отдельно
Замеры	2 процесса на разных ядрах, 10 и 3 повтора	3 процесса, минимум из 2–3 повторов
Метрика	сумма медиан CPU-времени	ср. геометрическое отношений
Интервал	бутстрэп по картам	бутстрэп по задачам
Проверка путей	стоимость, коллизии, восстановление пути	стоимость против всех вариантов